

Toward Scalable Analysis of Multidimensional Scientific Data: A Case Study of Electrode Arrays

Ye Niu[†], Abdullah Al-Mamun[‡], Hui Lin[‡], Tonglin Li[§], Yi Zhao[†], Dongfang Zhao^{†§}

[†]The Ohio State University [‡]University of Nevada, Reno [§]Lawrence Berkeley National Lab [§]UC Davis

Abstract—Many modern scientific applications involve large volumes of multidimensional data and extensive computation. Although distributed systems and tools are becoming increasingly scalable, they are still far away to catch up the exponential growth rate exhibited by many of those scientific big-data applications. This paper presents our early effort on overcoming the exponential complexity of one widely deployed workload over multidimensional scientific data—the $n \times n$ numerical analysis on two-dimensional arrays. More specifically, we propose a new approach to reduce the exponentially-grown data into a semantically-equivalent polynomial form in the context of two-dimensional electrode arrays, which are widely used in biomedical engineering, electrical engineering, and mechanical engineering. We have implemented a system prototype in Python, preliminary results show that the proposed approach outperforms the state-of-the-practice in various metrics: (i) the consumed space is six orders of magnitude smaller; (ii) the execution time is three orders of magnitude faster; and (iii) the scalability is improved by two orders of magnitude—from 6×6 to 100×100 —on mainstream servers in reasonable time.

Index terms—Scientific Computing; Multidimensional Data; Parallel Computing

I. INTRODUCTION

Many scientific applications take multidimensional arrays as the default format to store and manage large volumes of collected or simulated data [1]. In neurosciences, the three-dimensional signals of scanning a human brain are recorded at different timestamps and saved as four-dimensional arrays [2]. In astronomy, the telescope takes the pictures of the sky (a.k.a., sky surveys) periodically, resulting in a series of two-dimensional arrays, e.g., the LSST project [3]. In biomedical engineering, sets of micro-electrode grids are applied to tissues or cells to find the anomaly regions, and the data collected from the electrode wires are usually abstracted as two-dimensional arrays [4]. Because of the multidimensional nature of these scientific data, which are further complicated by the increasing volumes (e.g., the data are collected more frequently, in a higher resolution, and so forth), the analysis over these data imposes unique challenges than textual or flat-format data, and usually requires the user to have in-depth knowledge from both computer systems and domain sciences.

There indeed exist several popular computer systems designed for multidimensional data, sometimes called array databases. SciDB [5], for example, was originated from M.I.T. and is still under active development to store and analyze scientific data in multidimensional formats. Other systems are also available, such as RasDaMan [6] and SciSpark [7]; a comprehensive review can be found in our recent work [8].

One common challenge shared by all these array database systems, as noted by the paper [8], lies on the memory management because those scientific workflows can quickly use up available memory, leading to out-of-memory errors and application crashes. While memory resources might be technically easy to acquire in cloud computing platforms, scaling memory usually incur additional, sometimes unexpectedly and prohibitively high, financial cost [9]. The fundamental problem here is that the advances of modern computer hardware and software cannot catch up the rate at which the scientific applications generate the data. As a consequence, although many distributed systems and tools have emerged—not only the aforementioned array databases but also other general-purpose systems such as Spark [10] and Myria [11], the data-intensiveness nature of many scientific applications is still one of the most challenging problems in practice. If we could, somehow, reduce the complexity of the scientific applications, then the pressure on the computer system side would be greatly released and ultimately, the applications can be completed with both higher reliability and higher performance.

This paper presents our early effort on overcoming the exponential complexity, in both space and time, of one widely deployed workload over multidimensional scientific data—the $n \times n$ numerical analysis on multidimensional arrays. More specifically, we propose a new approach to reduce the exponentially-grown data into a semantically-equivalent polynomial form in the context of two-dimensional electrode arrays, which are widely used in biomedical engineering, electrical engineering, and mechanical engineering. To demonstrate the effectiveness of the proposed approach, we first prove the theoretical correctness, before implementing a system prototype and experimentally evaluating the performance. Preliminary results show that the proposed approach significantly outperforms the state-of-the-practice in various metrics: (i) the data size is six orders of magnitude smaller; (ii) the execution time is three orders of magnitude faster; and (iii) the scalability is improved by two orders of magnitude—from 6×6 to 100×100 —on mainstream hardware (Microsoft Azure Cloud [12]) in reasonable time (in the order of hours).

To summarize, this paper makes the following contributions:

- We propose a new methodology to overcome the compute-intensive problem in finding all the pair-wise paths in multidimensional data;
- We formally prove the correctness of a new algorithm with orders of magnitude less complexity in space, time, and scalability; and

- We experimentally verify the efficiency of the proposed method on mainstream computing platforms.

In the remainder of this paper, we review the literature in Section II. Section III formulates the problem from the multidimensional data challenge exemplified by the electrode arrays. We present the new methodology for addressing the big data challenge in Section IV. Section V reports the preliminary data we have collected from the experiments deployed to the Microsoft Azure Cloud. We finally conclude this paper in Section VI.

II. RELATED WORK

There is a rich literature for improving the analysis of the multidimensional data. To address the scalability challenge of power-law graphs, a two-dimensional partitioning algorithm [13] is proposed that combines graph partitioning with two-dimensional block distribution. A framework [14] is proposed to achieve wavefront parallelization that generates a runtime inspector responsible for collecting data-dependence information with less time and space overhead. In [15], a new approach is proposed to partition the matrix computation through MapReduce [16]. There is extensive work on tree codes and their parallelization for kernel summations in three dimensions, but high-dimensional problems remain challenging [17]. To overcome the challenge of superposition of high-level noises and interference, a hierarchical parallel processing framework over a GPU cluster [18] is developed to enable scalable factorization of large tensors in a divide-and-conquer. Multidimensional data processing is also actively researched in I/O subsystems and high-performance computing [19–27]. Although the aforementioned approaches address various problems with multidimensional array data in different domains, none of them proposes a systematical method to reduce the application's high complexity in space and time, by orders of magnitude, into a semantically-equivalent form.

The multidimensional electrode array problem is also being actively studied in other areas such as electrical power systems, which manipulate power flows among substations interconnected via electrical lines based on the measurements of power injections, voltage, or current [28]. To reduce the latency of solving non-linear equations in power systems, recent literature proposed to leverage the latest computer hardware and ad-hoc algorithms optimized for the new hardware, e.g., high-performance computers [29], GPUs [30]. However, the approaches from electrical power systems cannot be directly applied to multidimensional electrode arrays because those techniques assume the voltage and current can be measured, which is not the case for the multidimensional electrode arrays.

III. PROBLEM STATEMENT

A typical two-dimensional electrode array device comprises a set of horizontal wires and another set of vertical wires, both of which are connected through point-wise resistors. The orders of n 's highly depend on the specific applications. For instance, a device applied to a wound for anomaly detection may have $n = O(10)$, while a device designed for cell research

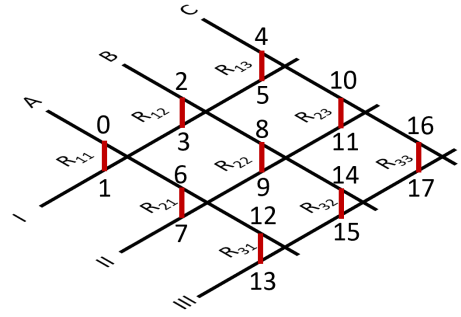


Figure 1. **Abstract Architecture of a Two-dimensional 3×3 Electrode Array.** Three horizontal wires (A, B, and C) and three vertical wires (I, II and III) are interconnected through the 18 joints $\{0, \dots, 17\}$ and nine resistors (R_{ij} in red, $1 \leq i, j \leq 3$).

requires at least $n = O(100)$. In general, a $n \times n$ array consists of $2n^2$ joints and n^2 resistors.

Figure 1 illustrates a concrete example of such a system where $n = 3$. Three horizontal wires (A, B, and C) and three vertical wires (I, II and III) are interconnected through the 18 joints $\{0, \dots, 17\}$ and nine resistors (R_{ij} in red, $1 \leq i, j \leq 3$). While the resistors can be, arguably, considered as the third dimensional in the abstract architecture, we usually do not count them because they are considered as scalars in the numerical analysis.

A common workload of such a $n \times n$ electrode array system is to find out the unknown R_{ij} 's ($1 \leq i, j \leq n$) given the pair-wise measured resistance between the end-points of n horizontal and n vertical wires. Therefore, we have n^2 unknowns (R 's) and also n^2 measured values, which are denoted as Z 's, namely Z_{AI} , Z_{BI} , Z_{CI} , Z_{AII} , Z_{BII} , Z_{CII} , Z_{AIII} , Z_{BIII} , and Z_{CIII} in the example shown in Figure 1. The challenge here is that the measured resistance between two end points is a nonlinear function of all the n^2 unknowns through many possible paths; To see this, take Z_{AI} for example, the most straightforward circuit is through R_{11} (between end points 0 and 1). And yet, there are other circuits flowing through, one possible path being:

$$A \rightarrow 6 \xrightarrow{R_{21}} 7 \rightarrow 9 \xrightarrow{R_{22}} 8 \rightarrow 2 \xrightarrow{R_{12}} 3 \rightarrow I$$

For a $n \times n$ array, there are overall $n^{(n+1)}$ possible paths. To see this, we can start with a specific pair of end points. Whenever the circuit flows from one joint to the next step, there are n possible choices. In total, there are only $(n - 1)$ steps between the source and the destination. Therefore, there are $n^{(n-1)}$ possibilities between any pair of end points. Note that there are totally n^2 pairs of end points. Consequently, the total number of paths is $n^{(n-1)} \cdot n^2 = n^{(n+1)}$. To save all of these possible paths, the required space is even larger than the n exponential because each path has to store all the joint numbers as well. We will report the numbers in the evaluation section; as we will see, the data growth is so fast that the path-based approach is unfeasible on mainstream computer hardware and systems when $n > 6$.

If we assume the paths can be stored efficiently (which is true only for small n 's), then the question is how to

find those paths efficiently. This is a classical problem in graph theory, which is solvable using either depth-first or breadth-first recursive traversal algorithms. Since the number of possible paths is exponential, any algorithm for finding them must be at least exponential, which is indeed the case of both depth-first and breadth-first recursive traversal algorithms. To see this, again, in the case of breadth-first recursion, there are n neighbors to the current position and each of the n neighbors might lead to a depth of $(n - 1)$, resulting in the total of exponential number of recursion calls.

The last step, after being able to find out and store the paths, is to solve the equations built up on the paths satisfying the constraints, i.e., the total incoming circuit flow is equal to the total outgoing circuit flow according to the Kirchhoff Law. In essence, all the paths are considered as the parallel circuit flows between two end points and can be aggregated through this form:

$$\frac{1}{Z_{ij}} = \sum_{k=1}^{n^{(n-1)}} \frac{1}{P_k(R)}$$

where $P_k(R)$ indicates the summation of resistors along the k -th path between the i -th horizontal wire and the j -th vertical wire in a $n \times n$ array. Therefore, the goal is to solve a system of n^2 nonlinear equations, each of which comprises an exponential number of terms, and each term exhibits a summation of selected unknowns as the divisor. This equation-solving procedure itself is also compute-intensive, requiring iterative method to find estimated roots.

IV. METHODOLOGY

A. Eliminating Redundant Sub-paths

To overcome the fundamental challenge in exponential compute and storage complexities, we propose to a new approach that is not tightly correlated to the $n^{(n+1)}$ paths between end points. Instead, we focus only on the n^2 joints and try to map a set of paths into a set of joints in the sense that all the topological properties are retained. In other words, we hope to reduce the number of constraints from $O(n^n)$ to $O(n^c)$ ¹ without losing any information, i.e., lossless conversion. Such a conversion is only possible if we could somehow express the partially redundant paths to a single joint. The key idea is inspired by the observation that many distinct end-to-end paths take the same sub-paths for a lot of times. Therefore, we attempt to reconstruct the $n \times n$ array into a different graph topology. We will first present the protocol of constructing the new graph using a real example with $n = 3$, followed by a formal proof on the semantic equivalence between the new graph and the original array structure.

Figure 2 illustrates a concrete example for converting all possible paths between two end points A and I in Figure 1. In the last section, we discuss the total number of possible paths between a specific pair of end points: $n^{(n-1)}$, in this case $3^{(3-1)} = 9$ paths. To validate the equality between

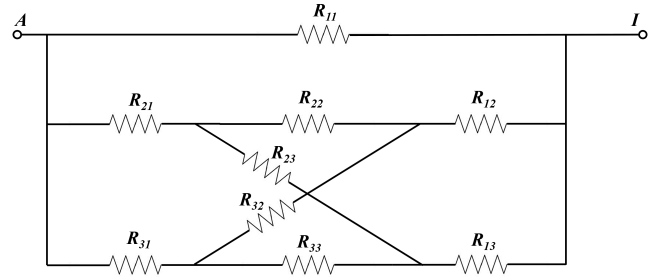


Figure 2. **An Equivalent Topology between A and I** as Figure 1. There are also nine paths between A and I , semantically equal to Figure 1 but with possible loops across sub-paths.

the transformed topology and the original array, we find the following nine paths from A to I (a formal proof will follow in a more general form):

- (i) $A \rightarrow R_{11} \rightarrow I$
- (ii) $A \rightarrow R_{11} \rightarrow R_{12} \rightarrow R_{32} \rightarrow R_{33} \rightarrow R_{13} \rightarrow I$
- (iii) $A \rightarrow R_{11} \rightarrow R_{13} \rightarrow R_{23} \rightarrow R_{22} \rightarrow R_{12} \rightarrow I$
- (iv) $A \rightarrow R_{21} \rightarrow R_{22} \rightarrow R_{12} \rightarrow I$
- (v) $A \rightarrow R_{21} \rightarrow R_{23} \rightarrow R_{13} \rightarrow I$
- (vi) $A \rightarrow R_{21} \rightarrow R_{22} \rightarrow R_{32} \rightarrow R_{33} \rightarrow R_{13} \rightarrow I$
- (vii) $A \rightarrow R_{31} \rightarrow R_{33} \rightarrow R_{13} \rightarrow I$
- (viii) $A \rightarrow R_{31} \rightarrow R_{32} \rightarrow R_{12} \rightarrow I$
- (ix) $A \rightarrow R_{31} \rightarrow R_{33} \rightarrow R_{23} \rightarrow R_{22} \rightarrow R_{12} \rightarrow I$

For an arbitrary pair of end points between the i -th horizontal wire and the j -th vertical wire, the equivalent topology can be expressed as Figure 3. In essence, the most straightforward path between i and j goes only through R_{ij} , shown as the top path (or the top main route) in the figure. Then, there are $(n - 1)$ main routes starting with R_{ik} where $k \in \{1, \dots, j - 1, j + 1, \dots, n\}$, corresponding to the first set of voltage values called $U_{a_{ijk'}}$ where $k' = k$ if $k \leq j$ and $k' = (k - 1)$ otherwise. Similarly, toward the end of each main route, there are $(n - 1)$ R_{mj} 's where $m \in \{1, \dots, i - 1, i + 1, \dots, n\}$ and $(n - 1)$ $U_{b_{ijm'}}$'s where $m' = m$ if $m \leq i$ and $m' = (m - 1)$ otherwise. We do not assign a variable to the end-to-end voltage between i and j (i.e., U_{ij}) because it can be easily measured in practice. Both U_a and U_b have three subscripts, with i and j indicating the two end points and the third subscript indicating the top-down ordering of those voltage values from 1 to $(n - 1)$. As we will see soon, this equivalent topology would yield a polynomial number of equations by enforcing the constraints on those $2(n - 1)$ voltage points U_a 's and U_b 's, as opposed to an exponential number of equations as discussed before. But before that, we need to prove the general topology presented in Figure 3 is equivalent to the original $n \times n$ array; that is to say, the new topology is a lossless transformation with all possible paths enclosed.

Theorem 1. *All paths from the i -th horizontal axis to the j -th vertical axis in a $n \times n$ array can be expressed by the transformed topology in Figure 3.*

Proof. We will prove the equivalence by induction.

It is obvious to see the topology is equivalent for $n = 1$ and $n = 2$. For $n = 3$, we demonstrate all the nine paths between

¹ C indicates a constant.

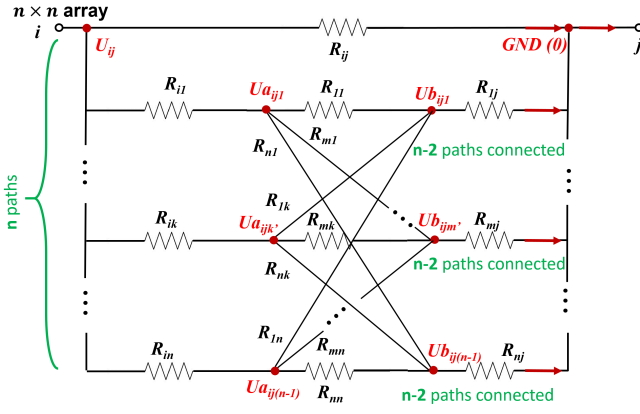


Figure 3. An Equivalent Topology between the i -th horizontal axis and the j -th vertical axis in a $n \times n$ array. There are total $(n-1)$ U_a 's and $(n-1)$ U_b 's as two sets of distinct voltage values for a specific pair of ij end points, assuming the original voltage can be measured between i and j . The conditions of subscripts are: $k \in \{1, \dots, j-1, j+1, \dots, n\}$; $m \in \{1, \dots, i-1, i+1, \dots, n\}$; $k' = k$ if $k \leq j$, $k' = (k-1)$ otherwise; and $m' = m$ if $m \leq i$, $m' = (m-1)$ otherwise.

one pair of end points A and I can be formed in the new topology as shown in Figure 2. Because all the horizontal and vertical axes are commutative along the same dimension, we can similarly build up nine paths for other eight combinatorial pairs of end points. Therefore, the new topology is indeed lossless at small scales, $n \in \{1, 2, 3\}$.

Now, we assume the new topology is lossless at a general rank $n > 3$, and we will prove the topology is still lossless at the higher rank $(n+1)$.

Before showing the paths are all retained at $(n+1)$, we need to guarantee no individual resistor is missing. That is, the new topology should consist of $(n+1)^2$ individual R 's. Following the general structure shown in Figure 3, the new $(n+1)$ -th main route will introduce three new R 's, $(n-1)$ R 's between the new U_a and all other main routes except for the top path through R_{ij} , and another $(n-1)$ R 's between the new U_b and all other main routes except for the top path. In other words, increasing n to $(n+1)$ introduces total $3 + 2 \cdot (n-1) = (2n+1)$ R 's. Therefore, the total number of R 's is $n^2 + (2n+1) = (n+1)^2$, which is exactly the expected number of R 's for a $(n+1) \times (n+1)$ array.

We will show the paths are all included as well for $(n+1)$. Recall that the total number of paths between any pair of end points is $n^{(n-1)}$; therefore, we expect the total number of paths is $(n+1)^n$ for a $(n+1) \times (n+1)$ array. If we think again how the paths are found in the new topology from a $n \times n$ array, we start from i and go through the n main routes (passing one R at each main route), and then encounter another n choices: either choosing one R from the $(n-1)$ neighbor R 's or stay at the current route (without passing any of the neighbor R 's). Such procedure is repeated for $(n-1)$ times, every iteration being one step closer to the destination j . In the transformed topology for rank $(n+1)$, the first step has one more possible path, i.e., the $(n+1)$ -th main route. Then, each of the following steps decides either to go through one of the n R 's or not to pass any at all (i.e., stay at the same

wire). Because there are total $(n+1)$ main routes, the total number of steps is n . Therefore, the transformed topology at rank $(n+1)$ exhibits all the $(n+1)^n$ possible paths from the original $(n+1) \times (n+1)$ array. $\square$

B. Building Up the System of Nonlinear Equations

Given the equivalent topology, we are able to enforce the constraints on the joints $(i, j, U_a$'s, and U_b 's) instead of the paths. The saving is significant: for each pair of end points, there are $2n$ joints (1 at i , 1 at j , $(n-1)$ at U_a 's, and $(n-1)$ at U_b 's) and $n^{(n-1)}$ paths; or for the entire system, there are a polynomial number $2n \cdot n^2 = O(n^3)$ of joints and exponential number $n^{(n-1)} \cdot n^2 = O(n^n)$ of paths. The following of this section explains how we generate the equations on those $2n$ joints for a pair of end points i and j .

The $2n$ equations (satisfying the circuit flow constraints by the Kirchhoff Law) for each pair of end points $\langle i, j \rangle$ are defined as follows:

$$\begin{cases} \frac{U_{ij}}{Z_{ij}} = \frac{U_{ij}}{R_{ij}} + \sum_k \frac{U_{ij} - U_{ijk'}}{R_{ik}} & \# \text{ One equation at } i \\ \frac{U_{ij}}{Z_{ij}} = \frac{U_{ij}}{R_{ij}} + \sum_m \frac{U_{ijm'}}{R_{mj}} & \# \text{ One equation at } j \\ \frac{U_{ij} - U_{ijk'}}{R_{ik}} = \sum_k \frac{U_{ijk'} - U_{ijm'}}{R_{mk}} & \# n-1 \text{ eq.'s for } U_a \\ \frac{U_{ijm'}}{R_{mj}} = \sum_m \frac{U_{ijk'} - U_{ijm'}}{R_{mk}} & \# n-1 \text{ eq.'s for } U_b \end{cases}$$

where (i) $k' = k$ if $k \leq j$ and $k' = (k-1)$ otherwise; and (ii) $m' = m$ if $m \leq i$ and $m' = (m-1)$ otherwise. For the entire array, there are $(n-1)n^2$ unknown U_a 's, $(n-1)n^2$ unknown U_b 's, and n^2 unknown R 's; all U_{ij} 's and Z_{ij} 's are measured values. The total number of nonlinear equations for the entire $n \times n$ array is $2n^3$, with $(2n-1) \cdot n^2$ unknowns. Although a system of nonlinear equations does not guarantee unique or sensible roots (for instance, resistance cannot be non-positive values), specifying a practical and positive U_{ij} value usually precludes the problem, which is out of the scope of this paper.

V. EVALUATION

We have implemented the proposed joint-constraint approach in Python and Bash script with about 1,200 lines of code (excluding the imported library such as `scipy.optimize.roots`). The source code is publicly accessible at <https://bitbucket.org/dongfang/mea>. Our test bed consists of two types of Microsoft Azure Cloud instances: Standard D16s v3 has 16 vCPUs, 64 GB RAM, a 1 TB SSD; Standard F64s v2 has 64 vCPUs, 128 GB RAM, and a 2 TB SSD. Both instance types are installed with Ubuntu 18.04.1, Python 2.7.15, and SciPy 1.1.0.

The first experiment quantitatively compares the data size of the generated equations from the conventional all-paths and the proposed joint-constraints, respectively. The experiment is carried out in the F64s instance on Microsoft Azure cloud. We report the results in Figure 4. At small scales $n \in \{2, 3\}$,

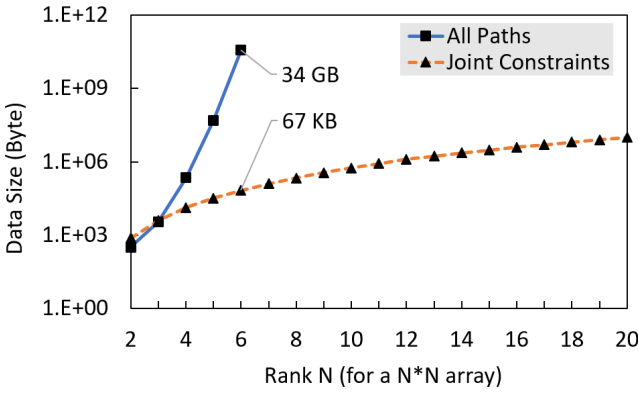


Figure 4. **Data size comparison between all-paths and joint-constraints.** All-paths approach generates equations at an exponential rate and run out of disk space at $n = 7$ (beyond 2 TB per node, estimated 800 TB per node based on the trend of $n \in \{2, \dots, 6\}$). Joint-constraints approach gracefully takes more disk space and still incurs less than 1 MB when $n = 20$.

both approaches generate similar volumes of data. However, all-paths approach generates equations at an steep exponential rate and run out of disk space at $n = 7$ (beyond 2 TB per node, estimated 800 TB per node based on the trend of $n \in \{2, \dots, 6\}$); In contrast, joint-constraints approach gracefully takes more disk space and still incurs less than 1 MB when $n = 20$. At the largest scale runnable for all-paths, $n = 6$, we observe that all-paths generate 34 GB data and joint-constraints generate only 67 KB data, making the proposed approach 6 orders of magnitude more space-efficient.

The second experiment compares the time to form those equations, either using all-paths or joint-constraints. We report the results for ranks $n \in \{2, \dots, 20\}$ in Figure 5. The experiment is carried out in the F64s instance on Microsoft Azure cloud. The time taken by all-paths increases at an exponential rate and surpasses one hour at the largest possible scale $n = 6$ (Recall that the all-paths method runs out of the space when $n = 7$). The joint-constraints method forms the equations instantly (i.e., less than one second) for $n \leq 10$ and only takes 20 seconds when $n = 20$. At the largest possible scale where the all-paths method works $n = 6$, the proposed joint-constraints approach (1 second) is thus three orders of magnitude faster the all-paths method (4,483 seconds).

The third experiment reports the temporal and spatial performance of the proposed joint-constraints method at larger scales. That is, we evaluate the space consumption (i.e., the generated data size) and the time taken for forming the equations beyond $n = 20$. The experiment is carried out in the F64s instance on Microsoft Azure. Figure 6 reports the results on the scalability of the proposed approach at up to $n = 100$. On a single node, the compute time and data size both gracefully grow and end with a 6.8 GB file for storing all the equations in 4.3 hours. One of our future work is thus to parallelize the formation of these constraint functions to reduce the overall compute time.

The fourth and last experiment is to verify whether the joint-constraints equations are solvable, in reasonable time, by numerical methods, e.g., pick an initial set of values

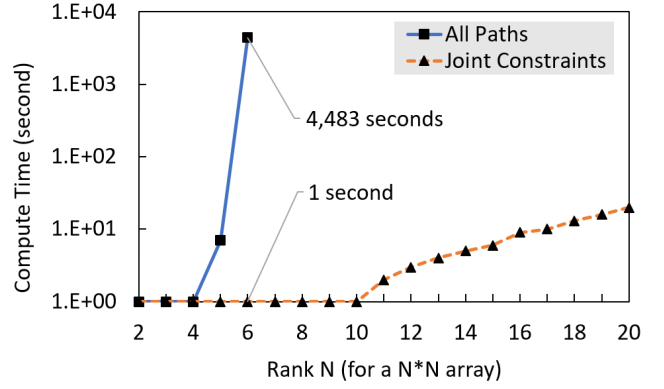


Figure 5. **Time comparison between all-paths and joint-constraints.** The time taken by all-paths increases at an exponential rate and surpasses one hour at the largest possible scale $n = 6$. Joint-constraints form the equations instantly (i.e., less than one second) for $n \leq 10$ and only takes 20 seconds when $n = 20$.

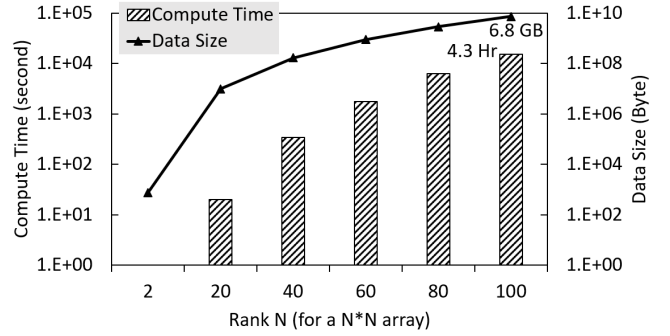


Figure 6. **Temporal and Spatial Scalability of Joint-constraints-based Equation Generation.** On a single node, the compute time and data size both gracefully grow and end up with a 6.8 GB file for storing all the equations in 4.3 hours.

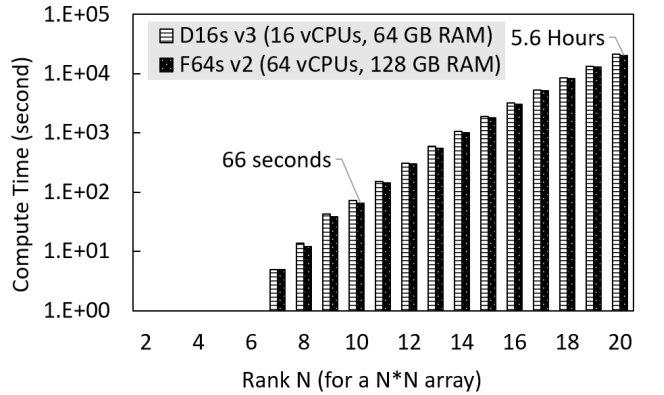


Figure 7. **Solving Time of the Joint-constraints Equations on Two Types of Machines.** Different machines exhibit negligible differences on the time taken to solve the equations at various scales between $n = 2$ and $n = 20$. At $n = 10$, both machines can solve the equations in roughly a minute; at $n = 20$, both machines can solve them in about 5.6 hours. Times shorter than one second are not shown in the figure.

and repeatedly adjust them to converge to the true values satisfying the equations. We deploy the implementation on two different machine types—D16s v3 and F64s v2 on Microsoft Azure Cloud, and apply `scipy.optimize.roots` with the Levenberg-Marquardt algorithm (i.e., `method = 'lm'`) to

the equations. We report the solving time in Figure 7. As we can see, different machines exhibit negligible differences on the time taken to solve the equations at various scales between $n = 2$ and $n = 20$; this is because the Levenberg-Marquardt algorithm is not fully parallelized to take advantage of the many cores in the machine. At $n = 10$, both machines can solve the equations in 66 seconds; at $n = 20$, both machines can solve them in about 5.6 hours.

VI. CONCLUSION AND FUTURE WORK

This paper presents our early effort on overcoming the exponential complexity, in both space and time, of one widely deployed workload over multidimensional scientific data—the $n \times n$ numerical analysis on multidimensional arrays. We prove the theoretical correctness of the proposed approach, and experimentally evaluate its performance. Preliminary results show that the proposed approach significantly outperforms the state-of-the-practice in space, time, and scalability: the consumed space is six orders of magnitude smaller; the execution time is three orders of magnitude faster; and the scalability is improved by two orders of magnitude—from 6×6 to 100×100 .

In the future, we will parallelize the procedure to generate the large-scale nonlinear equations and reduce the root space for the equations. One challenge to achieve that is the potentially infinite number of roots for the system of nonlinear equations generated from large multidimensional data. A possible solution is to exploit domain-specific knowledge to eliminate the unqualified roots at early stages (early pruning), which will be investigated in our future work. In addition, we will also explore new methodology to solve the compute-intensive multidimensional-data management challenges through decentralized approaches such as high-performance blockchains [31].

Acknowledgement. This work is in part supported by a Microsoft Azure Research Award.

REFERENCES

- [1] D. Zhao and L. Yang, “Incremental isometric embedding of high-dimensional data using connected neighborhood graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 31, no. 1, pp. 86–98, 2009.
- [2] F. Pestilli, J. D. Yeatman, A. Rokem, K. Kay, and B. Wandell, “Evaluation and statistical inference for human connectomes,” *Nature Methods*, vol. 11, no. 10, pp. 1058–63, 2014.
- [3] Large Synoptic Survey Telescope, “<https://www.lsst.org/>,” Accessed June 29, 2017.
- [4] Y. Niu, L. Qi, F. Zhang, and Y. Zhao, “Geometric screening of core/shell hydrogel microcapsules using a tapered microchannel with interdigitated electrodes,” *Biosensors and Bioelectronics*, vol. 112, pp. 162–169, 2018.
- [5] SciDB, “<https://paradigm4.atlassian.net/wiki/display/ESD/SciDB+Documentation>,” Accessed July 25, 2016.
- [6] Rasdaman, “<http://rasdaman.org/>,” Accessed Sept. 29, 2018.
- [7] SciSpark, “<https://scispark.jpl.nasa.gov/>,” Accessed Sept. 29, 2018.
- [8] P. Mehta, S. Dorkenwald, D. Zhao, T. Kaftan, A. Cheung, M. Balazinska, A. Rokem, A. Connolly, J. Vanderplas, and Y. AlSayyad, “Comparative evaluation of big-data systems on scientific image analytics workloads,” in *Proceedings of the 43rd International Conference on Very Large Data Bases (VLDB)*, 2017.
- [9] J. Wang, C. Xu, K. Wang, F. Yan, and D. Zhao, “Toward cost-effective memory scaling in clouds: Symbiosis of virtual and physical memory,” in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, 2018.
- [10] Apache Spark, “<http://spark.apache.org/>,” Accessed December 23, 2015.
- [11] Myria, “<http://myria.cs.washington.edu/>,” Accessed July 18, 2016.
- [12] Microsoft Azure, “<https://azure.microsoft.com/>,” Accessed Sept. 29, 2018.
- [13] E. G. Boman, K. D. Devine, and S. Rajamanickam, “Scalable matrix computations on large scale-free graphs using 2d graph partitioning,” in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*, 2013, pp. 50:1–50:12.
- [14] A. Venkat, M. S. Mohammadi, J. Park, H. Rong, R. Barik, M. M. Strout, and M. Hall, “Automating wavefront parallelization for sparse matrix computations,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, Nov 2016, pp. 480–491.
- [15] J. Xiang, H. Meng, and A. Aboulmaga, “Scalable matrix inversion using mapreduce,” in *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing (HPDC)*, 2014, pp. 177–190.
- [16] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” in *Proceedings of USENIX Symposium on Operating Systems Design & Implementation*, 2004.
- [17] W. B. March, B. Xiao, C. D. Yu, and G. Biros, “An algebraic parallel tree code in arbitrary dimensions,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2015, pp. 571–580.
- [18] D. Chen, Y. Hu, L. Wang, A. Y. Zomaya, and X. Li, “H-parafac: Hierarchical parallel factor analysis of multidimensional big data,” *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 28, no. 4, pp. 1091–1104, April 2017.
- [19] Z. Zhou, X. Yang, D. Zhao, P. Rich, W. Tang, J. Wang, and Z. Lan, “I/O-aware batch scheduling for petascale computing systems,” in *Cluster Computing, IEEE International Conference on*, 2015.
- [20] D. Zhao, J. Yin, K. Qiao, and I. Raicu, “Virtual chunks: On supporting random accesses to scientific data in compressible storage systems,” in *IEEE International Conference on Big Data*, 2014, pp. 231–240.
- [21] D. Zhao, J. Yin, and I. Raicu, “Improving the i/o throughput for data-intensive scientific applications with efficient compression mechanisms,” in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC '13), poster session*, 2013.
- [22] T. Li, C. Ma, J. Li, X. Zhou, K. Wang, D. Zhao, and I. Raicu, “Graph/z: A key-value store based scalable graph processing system,” in *Cluster Computing, IEEE International Conference on*, 2015.
- [23] T. Li, K. Keahey, K. Wang, D. Zhao, and I. Raicu, “A dynamically scalable cloud data infrastructure for sensor networks,” in *Proceedings of the 6th Workshop on Scientific Cloud Computing (ScienceCloud)*, 2015.
- [24] D. Zhao, Z. Zhang, X. Zhou, T. Li, K. Wang, D. Kimpe, P. Carns, R. Ross, and I. Raicu, “FusionFS: Toward supporting data-intensive scientific applications on extreme-scale distributed systems,” in *Proceedings of IEEE International Conference on Big Data*, 2014, pp. 61–70.
- [25] D. Zhao, K. Qiao, and I. Raicu, “Hycache+: Towards scalable high-performance caching middleware for parallel file systems,” in *Proceedings of the 14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, 2014, pp. 267–276.
- [26] D. Zhao, N. Liu, D. Kimpe, R. Ross, X.-H. Sun, and I. Raicu, “Towards exploring data-intensive scientific applications at extreme scales through systems and simulations,” *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 27, no. 6, pp. 1824–1837, June 2016.
- [27] L. Jiang, K. Wang, and D. Zhao, “Davram: Distributed virtual memory in user space,” in *IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, Accepted, 2018.
- [28] Y. Liao and M. Kezunovic, “Online optimal transmission line parameter estimation for relaying applications,” *IEEE Transactions on Power Delivery*, vol. 24, no. 1, pp. 96–102, Jan 2009.
- [29] A. C. B. Alves and A. Monticelli, “Parallel and distributed solutions for contingency analysis in energy management systems,” in *38th Midwest Symposium on Circuits and Systems. Proceedings*, vol. 1, Aug 1995, pp. 449–452 vol.1.
- [30] H. Karimipour and V. Dinavahi, “Accelerated parallel wls state estimation for large-scale power systems on gpu,” in *2013 North American Power Symposium (NAPS)*, Sept 2013, pp. 1–6.
- [31] A. Al-Mamun, T. Li, M. Sadoghi, and D. Zhao, “In-memory blockchain: Toward efficient and trustworthy data provenance for hpc systems,” in *IEEE International Conference on Big Data (BigData)*, 2018.